GitHub Workflow

How we work on ButtonCounter. Everything starts with an issue and ends with a reviewed PR merged into `development`.

Branches

Branch	Purpose
`main`	Stable. Release only. No direct commits.
`development`	Integration branch. All PRs target this.
`<issue-id>-<type>-<slug>`	Work branches. Named after the primary issue.

Rules:

- Never commit directly to `main` or `development`.
- Branch off the latest `development`.
- One branch, one PR.
- Delete the branch after merge.

Naming

Format: `<issue-id>-<type>-<slug>`

- `<issue-id>` is the GitHub issue number of the primary issue, no `#`.
- `<type>` is `feat`, `fix`, `chore`, `docs`, `refactor`, or `test`.
- `<slug>` is 2 to 4 lowercase words, hyphen separated.
- No uppercase, no underscores, no spaces.

Examples:

```
1-feat-setup-turso-database
14-feat-counter-reset-button
27-fix-login-redirect
```

Do not use `:` in a branch name. Git rejects it. Avoid `(` and `)` too, the shell fights you.

Create it:

```
git checkout development
git pull origin development
git checkout -b 14-feat-counter-reset-button
```

Multiple issues on one branch

A branch may cover more than one issue when the work is genuinely one unit, for example a feature and the small fix it depends on.

- Name the branch after the primary issue only.
- Give each issue its own commit or commits, tagged with its id.
- List every issue in the PR body so all of them close on merge.
- If the issues can ship separately, split them into separate branches instead.

Issues

Every change needs an issue first. No issue, no branch.

An issue should have:

- A clear title in plain language.
- A description: what the problem is, what "done" looks like.
- Labels: type (`bug`, `feature`, `chore`, `docs`) and priority if known.
- An assignee before work starts.

Keep issues small. If one issue needs more than a few days of work, split it.

Close issues through the PR, not by hand. Put `Closes #14` in the PR body and GitHub closes it on merge.

Commits

Format: `<type>: <what changed> (#<issue-id>)`

Types: `feat`, `fix`, `chore`, `docs`, `refactor`, `test`, `style`.

```
feat: add reset button to counter (#14)
fix: prevent negative count on rapid click (#15)
docs: add workflow guide (#16)
```

The trailing id is what maps a commit back to its issue on a shared branch, so do not skip it. Keep the subject under 72 characters, present tense, no trailing period. Commit small and often.

Pull Requests

Before opening

```
git checkout development
git pull origin development
git checkout 14-feat-counter-reset-button
git rebase development
npm run lint
npm run test
git push -u origin 14-feat-counter-reset-button
```

Fix conflicts on your branch, not in the PR.

Opening

- Base branch: `development`. Always.
- Title: same style as a commit subject.
- Body: what changed, why, how to test it, and a `Closes #<issue-id>` line per issue.
- Add screenshots for any UI change.
- Request at least one reviewer.
- Mark it a draft if it is not ready.

Template:

```
## What
Short summary of the change.

## Why
Closes #14
Closes #15

## How to test
1. npm run dev
2. Click the reset button
3. Count returns to 0

## Notes
Anything a reviewer should know.
```

Review

- At least one approval is required before merge.
- Reviewers respond within one working day.

- Comments are requests, not orders. Discuss if you disagree.
- Push fixes as new commits so the reviewer can see what changed.
- Resolve a thread only after the change is pushed.
- The author does not merge until approved and CI is green.

Merge

- Single issue branch: **Squash and merge** into `development`.
- Multi issue branch: **Rebase and merge**, so each issue keeps its own commit.
- Clean up the message before confirming.
- Delete the branch after merge.
- Do not merge your own PR without an approval.

Release to main

`main` is the deploy branch. Pushing to `main` triggers the deploy job, and the live URL always reflects what is on `main`. Nobody has to remember to ship before a demo.

Rules:

- The only way code reaches `main` is a PR from `development`. No other source branch.
- No direct pushes to `main`. Protect the branch in repo settings.
- Deploy runs only after the checks job passes. A red build never ships.
- Use a merge commit, not a squash, so release history is kept.

Steps:

1. Confirm `development` is green and stable.
2. Open a PR from `development` to `main`.
3. Get an approval. Same review rules as any other PR.
4. Merge. The deploy job runs on the push and updates the live URL.
5. Watch the run. If deploy fails, fix forward on a new branch, do not push to `main`.

Deploy

Defined in `.github/workflows/ci.yml`. Two jobs:

Job	Runs on	Does
`check`	Every PR, and every push to `main`	lint, check, test, build
`deploy`	Push to `main` only	build and publish

`deploy` declares `needs: check`, so a failing check blocks the deploy in the same run.

Secrets

Set these in Settings, Secrets and variables, Actions:

- `TURSO_DATABASE_URL`
- `TURSO_AUTH_TOKEN`

This repo is public. Anyone can read the workflow YAML, so a token in the YAML is a leaked token. Secrets only. GitHub masks them in logs as `***`. If you ever see a raw token in a run log, rotate it.

No preview deploys

Preview deploys are off, on purpose. A preview build pointing at the same Turso database increments the real global counter every time someone opens a PR. If we ever want previews, they need their own database and their own secrets:

```
turso db create button-counter-preview
```

Quick reference

```
issue -> branch (<issue-id>-<type>-<slug>) -> commits tagged (#id) -> PR to development ->
review -> merge -> delete branch
```